

FRESHROUTE: FRUIT SUPPLY CHAIN MANAGEMENT SYSTEM

Project Deployment Report

AWS Backend Deployment and React Native Frontend Release Plan

Project ID - 25-26J-444

Samarakoon A D	IT22124494
Dias N A	IT22213730
Gallearachchi S C	IT22230874
Dayarathna W S R	IT22257154

B.Sc. (Hons) in Information Technology
Specializing in Software Engineering
Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	4
1.1 Overview	4
1.2 Key Metrics	4
2. Deployment Objectives.....	4
2.1 Goals	4
2.2 Scope.....	4
3. Repository and System Overview	5
3.1 Repository Mapping.....	5
3.2 Expected System Flow	5
3.3 Repository Values to Confirm Before Deployment.....	5
4. Recommended AWS Architecture	5
4.1 Proposed Architecture	5
4.2 High-Level Architecture Diagram.....	6
4.3 Deployment Strategy Selection	6
5. Pre-Deployment Activities	6
5.1 Environment Preparation	6
5.2 Pre-Deployment Testing	7
5.3 Backup Plan.....	7
6. Backend Deployment on AWS EC2	7
6.1 Deployment Information	7
6.2 Step-by-Step Deployment Procedure	8
Step 1: Launch AWS EC2 Instance.....	8
Step 2: Configure Security Group Rules.....	8
Step 3: Connect to the EC2 Instance.....	8
Step 4: Update Server Packages	8
Step 5: Install Node.js and Package Manager.....	8
Step 6: Install PM2	8
Step 7: Clone the Backend Repository.....	8
Step 8: Configure Environment Variables.....	8
Step 9: Install Dependencies and Test Application.....	8
Step 10: Start Backend with PM2	9
Step 11: Optional PM2 Ecosystem File	9
Step 12: Configure Nginx Reverse Proxy.....	9
Step 13: Configure SSL/TLS with Certbot.....	9
Step 14: Configure Database Connectivity	9
Step 15: Set Up GitHub Actions CI/CD for Backend	10
Step 16: Verify Public API.....	10
7. React Native Frontend Deployment Strategy	11
7.1 Why React Native Is Different From Web Deployment	11

7.2 Recommended Frontend Release Options	11
7.3 Frontend Configuration Before Build.....	11
7.4 Expo / EAS Build Path.....	11
7.5 Bare React Native Android Release Path	11
7.6 Bare React Native iOS Release Path.....	12
7.7 Recommended Approach for FreshRoute	12
8. Tools and Technologies.....	12
9. Deployment Checklists.....	12
9.1 Pre-Deployment Checklist	12
9.2 Backend Deployment Checklist.....	13
9.3 Frontend Deployment Checklist	13
9.4 Post-Deployment Checklist.....	13
10. Issues Encountered and Resolutions	14
11. Performance, Monitoring, and Security	14
11.1 Performance Targets	14
11.2 Monitoring Plan	14
11.3 Security Controls.....	14
12. Rollback and Backup Plan	15
12.1 Backend Rollback	15
12.2 Frontend Rollback.....	15
12.3 Backup Strategy	15
13. Post-Deployment Activities	15
13.1 Validation and Testing	15
13.2 Documentation Updates.....	15
13.3 Training and Support	15
14. Lessons Learned and Recommendations.....	15
14.1 Success Factors	15
14.2 Areas for Improvement.....	16
14.3 Recommendations	16
15. Appendices	17
Appendix A: Backend Deployment Command Summary.....	17
Appendix B: React Native Build Command Summary.....	17
Appendix C: Evidence Screenshot Checklist	17
Appendix D: Reference Documentation	20

1. Executive Summary

1.1 Overview

FreshRoute is a fruit supply chain management system supported by two repositories: a backend API repository and a React Native mobile application repository. The deployment objective is to host the backend on AWS in a secure, stable, and maintainable production-style environment, while preparing the React Native frontend for tester distribution and later public release through mobile app stores.

This report follows the structure of a formal project deployment report and converts the current plan into an implementable AWS deployment procedure. Because the deployment has not yet been executed, this document is written as a planned deployment report with placeholders for deployment date, server public IP, domain name, database URI, and final branch names. These values must be replaced after AWS provisioning and repository verification.

1.2 Key Metrics

Metric	Planned Value / Status
Deployment Type	Planned production-style deployment
Backend Target Platform	AWS EC2 instance running Ubuntu 24.04 LTS or compatible Linux AMI
Backend Runtime	Node.js LTS version matching package.json / .nvmrc
Process Manager	PM2 with startup persistence
Reverse Proxy	Nginx
Public Access	HTTPS through domain such as https://api.freshroute.lk
Database	Use the backend repo configuration: MongoDB Atlas/AWS DocumentDB, RDS, or Firebase as applicable
Frontend Type	React Native mobile application
Frontend Release	Android APK/AAB for testing; Google Play / TestFlight for production
CI/CD	GitHub Actions via SSH deploy for backend; EAS Build or local Gradle/Xcode for frontend
Monitoring	PM2 logs, Nginx logs, CloudWatch metrics and alarms
Security	SSH key access, least-privilege IAM, HTTPS, restricted inbound ports, secret isolation

2. Deployment Objectives

2.1 Goals

1. Deploy the FreshRoute backend API on AWS with reliable service management and automatic restart capability.
2. Expose the API through HTTPS using Nginx reverse proxy and SSL/TLS certificates.
3. Configure repository secrets and environment variables securely without committing sensitive data to GitHub.
4. Prepare a repeatable GitHub Actions deployment pipeline for backend updates.
5. Define a clear deployment path for the React Native mobile application, including Android testing and app store release preparation.
6. Provide validation, monitoring, rollback, and backup procedures suitable for academic project demonstration and future production scaling.

2.2 Scope

- Included: AWS EC2 provisioning, Linux preparation, Node.js runtime setup, backend repository clone, environment configuration, PM2 service setup, Nginx reverse proxy, SSL setup, GitHub Actions deployment, monitoring, and validation.
- Included: React Native frontend build and release strategy for Android and iOS, including Expo EAS Build and bare React Native alternatives.

- Excluded: Actual payment configuration for app stores, final production domain purchase, final AWS billing setup, and any unavailable private repository secrets.
- Assumption: The final backend start command and environment variables will be confirmed from the backend repository package.json, README, and .env.example before live deployment.

3. Repository and System Overview

3.1 Repository Mapping

Repository	Component	Expected Responsibility
RP_FreshRoute_Backend	Backend API	Authentication, fruit supply chain records, business logic, database access, file uploads, notifications, and API endpoints.
RP_FreshRoute_Fruit-Supply-Chain	React Native frontend	Mobile user interface for farmers, suppliers, distributors, retailers, admins, and/or customers depending on implemented roles.

3.2 Expected System Flow

Layer	Description
Mobile Client	React Native application installed on Android/iOS devices. It calls the API through a production base URL.
API Gateway Layer	Nginx receives HTTPS requests and forwards them to the Node.js backend process running locally on the EC2 instance.
Application Layer	Node.js backend runs under PM2. The actual start script must match package.json.
Data Layer	Database service stores users, fruit batches, supply chain events, inventory, orders, status updates, and related records.
Storage Layer	If the app uses images/documents, use AWS S3 or the existing configured storage provider instead of storing files permanently on EC2.
Monitoring Layer	CloudWatch, PM2 logs, and Nginx logs support health checks and issue diagnosis.

3.3 Repository Values to Confirm Before Deployment

- Backend package manager: npm, yarn, or pnpm.
- Backend start command: npm start, npm run dev, npm run build && npm start, or a custom command.
- Runtime version: Node.js version from package.json engines, .nvmrc, README, or lockfile.
- Backend listening port: commonly 3000, 5000, or 8000; confirm from .env or server entry file.
- Database type and URI variable name: for example MONGO_URI, DATABASE_URL, DB_HOST, or Firebase credentials.
- Frontend environment variable used for API base URL: for example API_URL, BASE_URL, or EXPO_PUBLIC_API_URL.
- React Native type: Expo managed, Expo prebuild, or bare React Native CLI.

4. Recommended AWS Architecture

4.1 Proposed Architecture

Component	AWS / Tool Choice	Purpose
Compute	Amazon EC2	Hosts the backend API on a Linux server.
Operating System	Ubuntu 24.04 LTS or Amazon Linux 2023	Stable server operating system for Node.js backend deployment.

Reverse Proxy	Nginx	Routes public HTTPS traffic to the backend process running on localhost.
Process Management	PM2	Keeps backend running, restarts on crashes, and persists service after reboot.
DNS	Route 53 or external DNS provider	Maps api.freshroute.lk to the EC2 public IP.
SSL/TLS	Let's Encrypt Certbot or AWS ACM with load balancer	Encrypts traffic between mobile users and backend.
Database	MongoDB Atlas, Amazon DocumentDB, Amazon RDS, or Firebase	Stores application data according to backend implementation.
Object Storage	Amazon S3	Stores uploaded images/documents if the backend supports file uploads.
Monitoring	Amazon CloudWatch + PM2 logs	Collects server metrics, logs, and alerts.
CI/CD	GitHub Actions	Automates backend deployments after code changes.

4.2 High-Level Architecture Diagram

```

React Native Mobile App
|
| HTTPS API requests
v
api.freshroute.lk / EC2 Public IP
|
v
Nginx Reverse Proxy :80/:443
|
v
Node.js FreshRoute Backend on localhost:<APP_PORT>
|
+--> Database Service (MongoDB Atlas / DocumentDB / RDS / Firebase)
|
+--> Optional File Storage (AWS S3)
|
+--> Logs and Metrics (PM2, Nginx, CloudWatch)

```

4.3 Deployment Strategy Selection

- Recommended for this project phase: EC2 + PM2 + Nginx because it is simple, low-cost, easy to demonstrate, and similar to the sample deployment style.
- Recommended for later scaling: Dockerize the backend and migrate to AWS Elastic Beanstalk, ECS Fargate, or EKS after stabilizing the application.
- Recommended database approach: use managed database hosting to avoid data loss when EC2 instances are replaced or restarted.

5. Pre-Deployment Activities

5.1 Environment Preparation

- Create or confirm AWS account access and IAM user/role permissions for EC2, Security Groups, CloudWatch, S3, and Route 53 if required.
- Create an EC2 key pair or prepare an existing private key for SSH access.
- Decide the AWS Region. For Sri Lanka users, ap-south-1 (Mumbai) or ap-southeast-1 (Singapore) are common practical options.
- Prepare the backend environment variables using a secure local copy of .env or .env.production.
- Confirm database connection settings and whitelist the EC2 public IP if using a managed database such as MongoDB Atlas.
- Confirm the backend can run locally before deploying: install dependencies, start the API, and test health endpoints.
- Confirm the React Native app can run locally on Android emulator or device before release build generation.

5.2 Pre-Deployment Testing

- Backend unit tests and integration tests executed successfully, where available.
- Authentication APIs tested with valid and invalid credentials.
- Fruit supply chain workflows tested: create item/batch, update status, view history, and role-specific access.
- Database connection tested using production-like credentials.
- Frontend API base URL tested against a staging backend URL.
- Mobile app tested on at least one physical Android device and one emulator before release build.

5.3 Backup Plan

- GitHub repositories act as source-code backups.
- Environment variables are stored securely outside GitHub or in GitHub Actions secrets.
- Database backups or snapshots are enabled on the managed database provider.
- If EC2 is used, create an Amazon Machine Image (AMI) after successful setup for quick restoration.
- Rollback is possible by checking out the previous git commit and restarting PM2.

6. Backend Deployment on AWS EC2

6.1 Deployment Information

Metric	Planned Value / Status
Backend Repository	https://github.com/AashaniSamarakoon/RP_FreshRoute_Backend
Branch Deployed	main or selected deployment branch
AWS Region	ap-south-1 recommended; update based on actual AWS setup
EC2 Instance Type	t3.micro/t3.small for demo; t3.medium+ for heavier workloads
Operating System	Ubuntu 24.04 LTS or compatible Linux AMI
Application Port	[Confirm from backend .env/package.json]
Public API URL	https://api.freshroute.lk or http://15.206.81.206 for temporary testing
Service Name	freshroute-backend managed by PM2

6.2 Step-by-Step Deployment Procedure

Step 1: Launch AWS EC2 Instance

- Open AWS EC2 Console and launch a Linux instance.
- Recommended AMI: Ubuntu Server 24.04 LTS or Amazon Linux 2023.
- Select instance size based on expected usage. For academic demonstration, t3.micro or t3.small is usually sufficient. For production workloads, choose t3.medium or higher.
- Attach a key pair and create a security group with SSH restricted to your IP address.

Step 2: Configure Security Group Rules

- Allow SSH port 22 from your IP address only.
- Allow HTTP port 80 from 0.0.0.0/0 and ::/0.
- Allow HTTPS port 443 from 0.0.0.0/0 and ::/0.
- Do not expose the Node.js application port publicly after Nginx is configured. It should listen on localhost or be blocked by the security group.

Step 3: Connect to the EC2 Instance

- Use SSH with the private key generated during EC2 creation.

```
chmod 400 freshroute-key.pem
ssh -i freshroute-key.pem ubuntu@15.206.81.206
```

Step 4: Update Server Packages

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git curl build-essential nginx
```

Step 5: Install Node.js and Package Manager

Install the Node.js version required by the backend repository. If the project contains .nvmrc, use that version. Otherwise, install a supported Node.js LTS version.

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/master/install.sh | bash
source ~/.bashrc
nvm install --lts
nvm use --lts
node -v
npm -v
```

Step 6: Install PM2

```
sudo npm install -g pm2
pm2 -v
```

Step 7: Clone the Backend Repository

```
mkdir -p ~/freshroute
cd ~/freshroute
git clone https://github.com/AashaniSamarakoon/RP_FreshRoute_Backend.git backend
cd backend
git checkout main
```

Step 8: Configure Environment Variables

Create a production .env file in the backend project root. The exact variable names must match the backend implementation.

```
nano .env

# Example only - replace with actual variables from the backend repository
NODE_ENV=production
PORT=5000
DATABASE_URL=<database_connection_string>
JWT_SECRET=<strong_random_secret>
CORS_ORIGIN=*
S3_BUCKET=<optional_bucket_name>
AWS_REGION=ap-south-1
```

Step 9: Install Dependencies and Test Application

```
npm install
npm run build # Run only if the backend has a build script
```



```
npm start          # Or use the actual start command from package.json
```

In a second terminal, validate that the API responds locally:

```
curl http://localhost:<APP_PORT>/health
curl http://localhost:<APP_PORT>/
```

Step 10: Start Backend with PM2

Use the actual backend entry command. The following example assumes the API can be started with `npm start`.

```
pm2 start npm --name freshroute-backend -- start
pm2 status
pm2 logs freshroute-backend
pm2 save
pm2 startup
```

Step 11: Optional PM2 Ecosystem File

```
cat > ecosystem.config.js <<'EOF'
module.exports = {
  apps: [
    {
      name: 'freshroute-backend',
      script: 'npm',
      args: 'start',
      cwd: '/home/ubuntu/freshroute/backend',
      env: {
        NODE_ENV: 'production'
      },
      max_memory_restart: '500M',
      autorestart: true
    }
  ]
};
EOF
```

```
pm2 start ecosystem.config.js
pm2 save
```

Step 12: Configure Nginx Reverse Proxy

```
sudo nano /etc/nginx/sites-available/freshroute-api
server {
    listen 80;
    server_name api.freshroute.lk;

    location / {
        proxy_pass http://127.0.0.1:<APP_PORT>;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}
sudo ln -s /etc/nginx/sites-available/freshroute-api /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

Step 13: Configure SSL/TLS with Certbot

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d api.freshroute.lk
sudo certbot renew --dry-run
```

Step 14: Configure Database Connectivity

- If using MongoDB Atlas, add the EC2 public IP to the Network Access whitelist and set `DATABASE_URL` in `.env`.
- If using AWS RDS, place the database in a private subnet where possible and allow access only from the EC2 security group.
- If using Firebase, store service account credentials securely and avoid committing JSON credential files to GitHub.
- Run backend migrations or seed scripts only if the repository provides them.

Step 15: Set Up GitHub Actions CI/CD for Backend

Create `.github/workflows/deploy-backend.yml` in the backend repository. Configure repository secrets before enabling the workflow.

Secret Name	Purpose
AWS_EC2_HOST	EC2 public IP or DNS name
AWS_EC2_USER	SSH username, usually ubuntu
AWS_EC2_PRIVATE_KEY	Private SSH key content
APP_DIR	Backend directory on EC2, for example <code>/home/ubuntu/freshroute/backend</code>

```
name: Deploy FreshRoute Backend

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Deploy to AWS EC2
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.AWS_EC2_HOST }
          username: ${ secrets.AWS_EC2_USER }
          key: ${ secrets.AWS_EC2_PRIVATE_KEY }
          script: |
            cd ${ secrets.APP_DIR }
            git pull origin main
            npm install
            npm run build --if-present
            pm2 restart freshroute-backend
            pm2 save
```

Step 16: Verify Public API

```
curl https://api.freshroute.lk/health
curl https://api.freshroute.lk/
pm2 status
sudo tail -n 100 /var/log/nginx/error.log
```

7. React Native Frontend Deployment Strategy

7.1 Why React Native Is Different From Web Deployment

A React Native application is compiled into a mobile application package and installed on a device. Therefore, the frontend repository is not normally hosted on AWS EC2 unless it also contains a web dashboard. The usual deployment output is an Android APK/AAB, an iOS IPA, or an app store submission.

7.2 Recommended Frontend Release Options

Option	Use When	Output
Expo EAS Build	The project uses Expo or can be configured with EAS.	Cloud-built Android APK/AAB and iOS IPA, internal distribution link, app store submission.
Bare React Native Android Build	The project has android/ folder and is managed with React Native CLI.	Signed APK for testing or AAB for Google Play.
Bare React Native iOS Build	The project has ios/ folder and Apple Developer access is available.	IPA through Xcode Archive or EAS build; distributed via TestFlight.
Static Web Hosting	Only if the repo contains a React web dashboard or Expo web build.	S3 + CloudFront, Vercel, Netlify, or Amplify hosting.

7.3 Frontend Configuration Before Build

- Replace all localhost API URLs with the production API URL, for example `https://api.freshroute.lk`.
- Use environment files such as `.env.production` and keep secrets out of the mobile bundle. Mobile apps cannot safely hide secrets embedded in JavaScript.
- Check Android permissions, network security configuration, app icon, splash screen, package name, and version code.
- Check iOS bundle identifier, app icon, splash screen, usage descriptions, signing team, and provisioning profiles.

```
# Example .env.production for mobile app
API_BASE_URL=https://api.freshroute.lk
APP_ENV=production
```

7.4 Expo / EAS Build Path

Use this path if the project uses Expo or if the team chooses to use EAS Build for React Native.

```
npm install -g eas-cli
cd RP_FreshRoute_Fruit-Supply-Chain
npm install
eas login
eas init
eas build:configure

# Preview build for testers
eas build -p android --profile preview

# Production builds
eas build -p android --profile production
eas build -p ios --profile production
```

7.5 Bare React Native Android Release Path

```
cd RP_FreshRoute_Fruit-Supply-Chain
npm install
npx react-native doctor

# Generate upload key
keytool -genkeypair -v -storetype PKCS12 \
  -keystore freshroute-upload-key.keystore \
  -alias freshroute-key-alias \
  -keyalg RSA -keysize 2048 -validity 10000

# Build Android App Bundle for Play Store
npx react-native build-android --mode=release
```

```
# Output location
android/app/build/outputs/bundle/release/app-release.aab
```

7.6 Bare React Native iOS Release Path

- Requires macOS, Xcode, and Apple Developer Program access.
- Set Bundle Identifier and Signing Team in Xcode.
- Create an Archive from Xcode and upload to App Store Connect.
- Use TestFlight for internal testing before public release.

7.7 Recommended Approach for FreshRoute

- First priority: deploy the backend on AWS and obtain a stable HTTPS API URL.
- Second priority: configure the React Native app to use the production API URL and generate an Android test APK or AAB.
- Third priority: distribute the Android build to supervisors/testers through direct APK, Google Play Internal Testing, or EAS internal distribution.
- Fourth priority: prepare iOS TestFlight only if Apple Developer access and macOS build environment are available.

8. Tools and Technologies

Technology	Version / Detail	Purpose
AWS EC2	Ubuntu 24.04 LTS / Linux AMI	Backend hosting environment
Node.js	Project-compatible LTS	Backend runtime
npm / yarn / pnpm	Confirm from lockfile	Dependency management
PM2	Latest stable	Backend process management and auto-restart
Nginx	Ubuntu package	Reverse proxy and HTTP/HTTPS routing
Certbot	Let's Encrypt	SSL/TLS certificate automation
GitHub	Two project repositories	Source code hosting
GitHub Actions	SSH deploy workflow	Backend CI/CD deployment
React Native	Frontend framework	Mobile app development
Expo EAS Build	Optional / recommended if Expo-compatible	Cloud mobile app builds and distribution
Google Play Console	Internal testing / production	Android distribution
Apple TestFlight	Internal testing / beta	iOS distribution
CloudWatch	Metrics and logs	Monitoring and alerting
S3	Optional	Image/document storage

9. Deployment Checklists

9.1 Pre-Deployment Checklist

- AWS account access confirmed.
- EC2 key pair created and stored securely.
- Backend repository access confirmed.
- Frontend repository access confirmed.
- Backend package.json reviewed for start/build scripts.
- Required Node.js version confirmed.
- Production .env variables prepared.
- Database connection string prepared and tested.
- Security group planned: SSH, HTTP, HTTPS only.
- Domain or temporary public IP selected for API testing.

- React Native API base URL configuration identified.

9.2 Backend Deployment Checklist

- EC2 instance launched successfully.
- SSH connection tested.
- System packages updated.
- Git, Nginx, build tools installed.
- Node.js LTS installed and verified.
- PM2 installed and verified.
- Backend repository cloned.
- Correct branch checked out.
- .env file created on server.
- npm install completed successfully.
- Backend started locally and tested.
- PM2 process created and saved.
- PM2 startup configured.
- Nginx reverse proxy configured.
- Nginx configuration tested and restarted.
- SSL certificate configured after domain setup.
- Public API endpoint tested.
- GitHub Actions secrets configured.
- CI/CD workflow tested with a safe commit.

9.3 Frontend Deployment Checklist

- Production API base URL configured.
- Mobile app tested against AWS backend.
- Debug-only settings removed.
- Android app package name confirmed.
- Android signing key generated and stored securely.
- Android release APK/AAB generated.
- Release build installed and tested on physical device.
- Google Play internal testing track prepared if needed.
- iOS signing and TestFlight prepared if Apple Developer account is available.
- Version name and version code updated before release.

9.4 Post-Deployment Checklist

- Health endpoint returns success through HTTPS.
- Authentication flow tested from mobile app.
- Core fruit supply chain records can be created, updated, viewed, and deleted according to role permissions.
- Database writes confirmed in production database.
- Nginx access and error logs reviewed.
- PM2 logs reviewed for runtime errors.
- CloudWatch alarms configured for CPU, disk, and memory if CloudWatch Agent is installed.
- Backup and rollback procedure verified.
- Final deployment screenshots captured for evidence.

10. Issues Encountered and Resolutions

Potential Issue	Root Cause	Resolution
Backend runs locally but not on EC2	Missing environment variables or wrong Node.js version	Confirm .env values, install project-required Node.js version, run npm install again.
502 Bad Gateway from Nginx	Backend process stopped or proxy_pass port incorrect	Check pm2 status and ensure Nginx proxy_pass matches application port.
MongoDB connection fails	Database IP whitelist or URI error	Whitelist EC2 public IP and verify DATABASE_URL/MONGO_URI.
PM2 process disappears after reboot	PM2 startup not configured	Run pm2 save and pm2 startup, then execute generated startup command.
GitHub Actions SSH fails	Incorrect private key or security group blocking SSH	Update repository secrets and allow SSH from GitHub runner or use a controlled deploy approach.
Mobile app calls localhost	React Native localhost points to the phone/emulator, not EC2	Replace localhost with https://api.freshroute.lk in production environment configuration.
Android release build cannot call HTTP API	Cleartext HTTP is blocked or API lacks HTTPS	Use HTTPS endpoint before release or configure network security only for temporary testing.
CORS errors in web/admin clients	Backend CORS policy not configured for production origin	Set CORS_ORIGIN to the required domain or implement an allowed origins list.
Uploaded images disappear after redeploy	Files stored on EC2 local disk	Use S3 or persistent storage for uploads.

11. Performance, Monitoring, and Security

11.1 Performance Targets

Metric	Target
API Availability	99%+ for project demonstration period
Standard API Response Time	< 2 seconds for normal CRUD operations
Heavy Operations	< 5-10 seconds depending on file processing and network conditions
CPU Utilization	Average below 70% during normal usage
Memory Utilization	Average below 80%; use PM2 memory restart threshold if needed
Error Rate	< 1% for tested production endpoints

11.2 Monitoring Plan

- Use pm2 status for process health.
- Use pm2 logs freshroute-backend for application runtime logs.
- Use /var/log/nginx/access.log and error.log for request and proxy issues.
- Enable CloudWatch Agent for memory, disk, and application log collection if budget allows.
- Create CloudWatch alarms for high CPU, low disk space, and instance status check failure.

11.3 Security Controls

- Restrict SSH access to trusted IP addresses only.
- Open only ports 80 and 443 publicly after deployment.
- Store secrets in .env on the server and GitHub Actions secrets, not in GitHub code.
- Use HTTPS for all mobile-backend communication.
- Use least-privilege IAM policies for AWS services such as S3.
- Rotate JWT secrets, database passwords, and access keys if exposed.

- Avoid storing AWS access keys inside the mobile app because mobile bundles can be extracted.

12. Rollback and Backup Plan

12.1 Backend Rollback

```
cd /home/ubuntu/freshroute/backend
git log --oneline
git checkout <previous_commit_hash>
npm install
npm run build --if-present
pm2 restart freshroute-backend
```

12.2 Frontend Rollback

- For APK distribution, redistribute the previous tested APK if the latest build fails.
- For Google Play, halt staged rollout and promote the previous stable build if available.
- For Expo Updates, publish a rollback update to the previous stable runtime channel if configured.
- For TestFlight, keep previous builds available for testers until the latest build is confirmed stable.

12.3 Backup Strategy

- Take database backups/snapshots before major deployment changes.
- Create an EC2 AMI after a stable backend deployment is completed.
- Export or securely store production .env values separately.
- Keep Android keystore files in secure storage; losing the keystore can block future updates.

13. Post-Deployment Activities

13.1 Validation and Testing

- Verify API health endpoint through HTTPS.
- Run end-to-end mobile tests: login, create supply chain entry, update status, retrieve details, and logout.
- Confirm production database records are created correctly.
- Verify image upload and retrieval if the app supports images.
- Review server and application logs after test execution.

13.2 Documentation Updates

- Update README files with production deployment instructions.
- Document final API base URL and environment variable list.
- Document the GitHub Actions deployment workflow and required secrets.
- Capture screenshots of successful AWS EC2 instance, PM2 status, Nginx status, SSL certificate, GitHub Actions run, and mobile release build.

13.3 Training and Support

- Provide team members with the deployment commands and rollback procedure.
- Train team members to check PM2 logs and Nginx logs for issue diagnosis.
- Share frontend release process with the member responsible for mobile builds.

14. Lessons Learned and Recommendations

14.1 Success Factors

- Separating backend hosting from mobile app distribution reduces deployment confusion.
- Using Nginx and PM2 provides a stable and understandable backend deployment model.
- A stable HTTPS API URL is essential before building the final React Native release.
- Keeping secrets outside GitHub reduces the risk of credential leakage.

14.2 Areas for Improvement

- Containerize the backend using Docker for easier migration across environments.
- Add automated backend tests to the CI/CD pipeline before deployment.
- Add a /health endpoint if not already available.
- Add centralized monitoring and alerting before production use.

14.3 Recommendations

- Use EC2 + PM2 + Nginx for the first deployment because it is simple and easy to document.
- Use a managed database rather than installing a database directly on EC2 for better reliability.
- Use Google Play Internal Testing or Expo EAS internal distribution for the first React Native release.
- Use AWS S3 for any uploaded images or files.
- Move to ECS Fargate or Elastic Beanstalk if the project later needs auto-scaling and zero-downtime deployments.

15. Appendices

Appendix A: Backend Deployment Command Summary

```
# Server preparation
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl build-essential nginx

# Node.js
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/master/install.sh | bash
source ~/.bashrc
nvm install --lts
nvm use --lts

# PM2
sudo npm install -g pm2

# Application
mkdir -p ~/freshroute
cd ~/freshroute
git clone https://github.com/AashaniSamarakoon/RP_FreshRoute_Backend.git backend
cd backend
nano .env
npm install
npm run build --if-present
pm2 start npm --name freshroute-backend -- start
pm2 save
pm2 startup
```

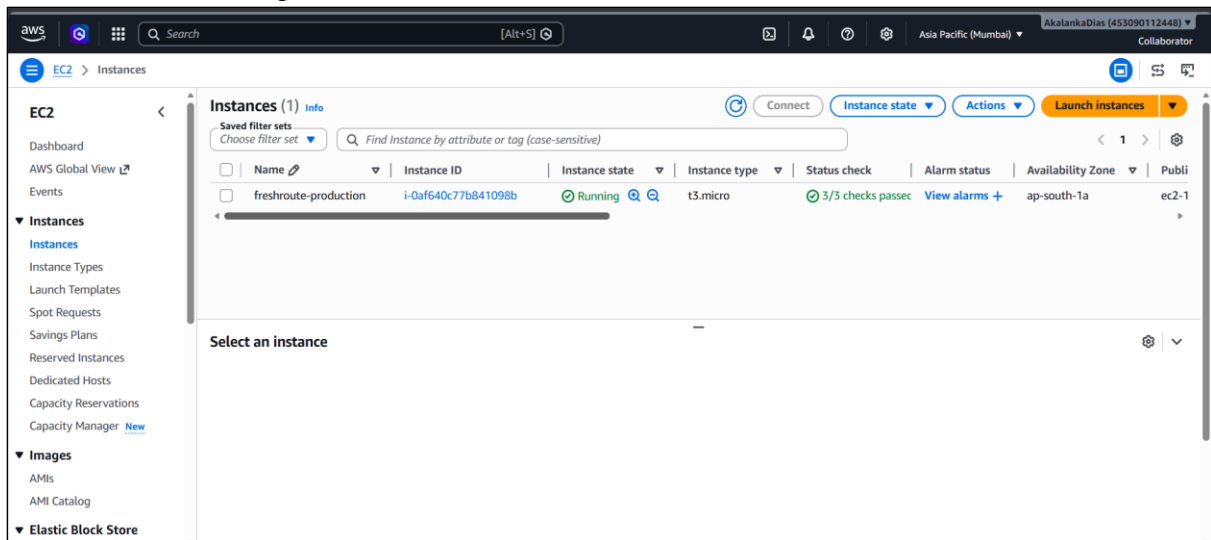
Appendix B: React Native Build Command Summary

```
# Expo / EAS path
npm install -g eas-cli
npm install
eas login
eas init
eas build:configure
eas build -p android --profile preview
eas build -p android --profile production

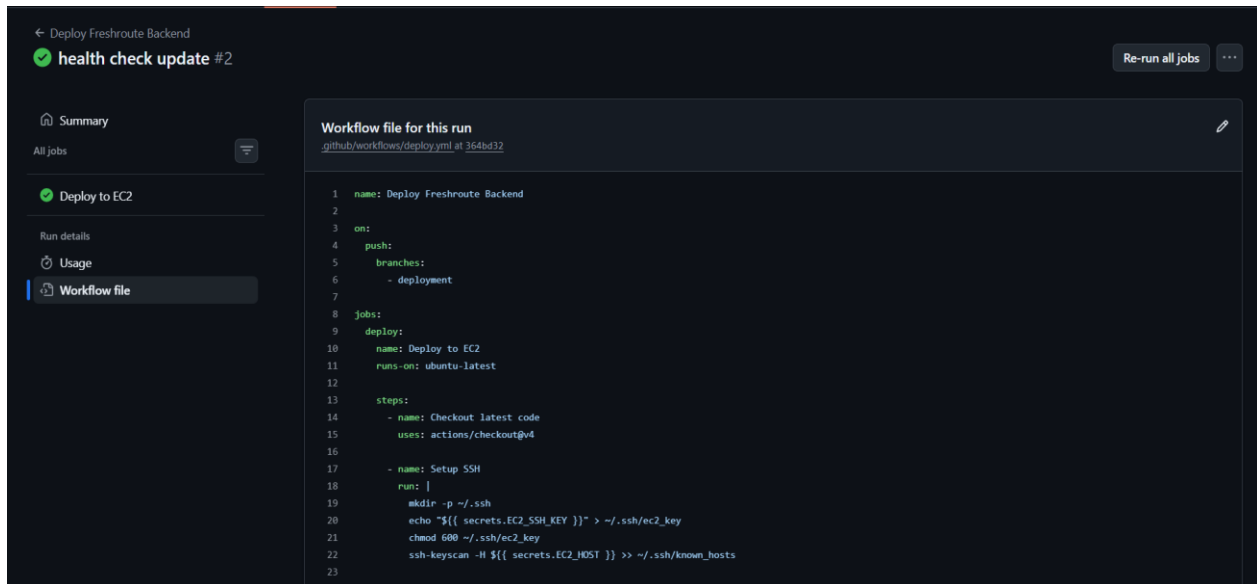
# Bare React Native Android path
npm install
npx react-native doctor
npx react-native build-android --mode=release
```

Appendix C: Evidence Screenshot Checklist

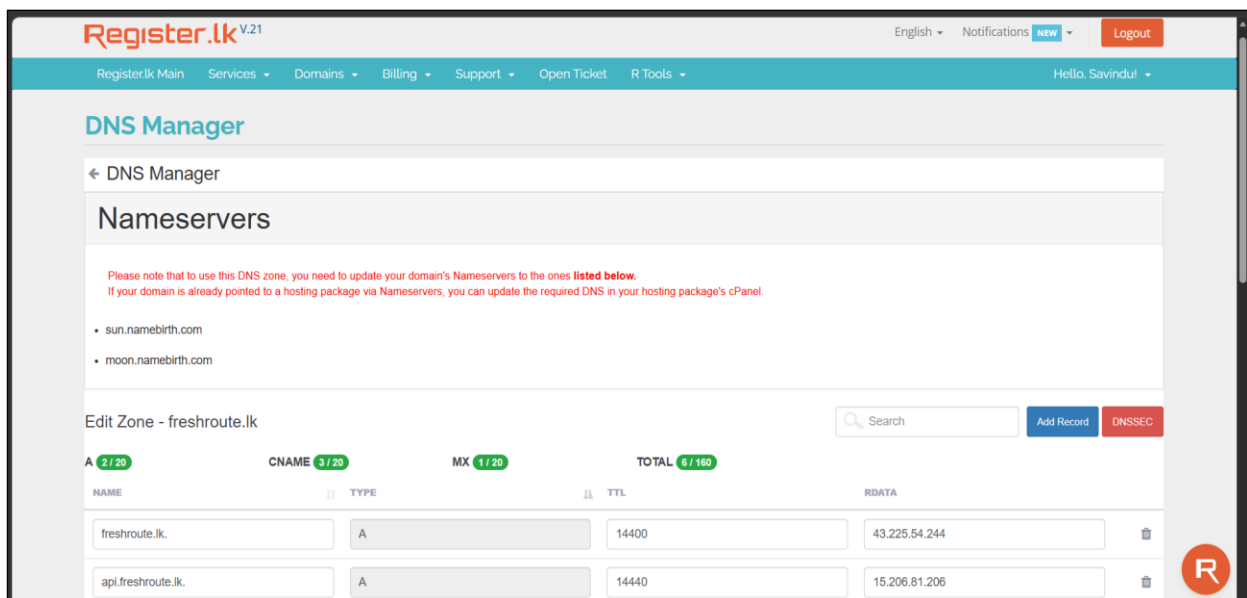
- AWS EC2 instance running status.



- EC2 CI/CD Workflow



- DNS Manager



- PM2 status showing freshroute-backend online.

```

id   name           namespace    version    mode    pid    uptime    Ⓢ    status    cpu    mem    user    watching
0    freshroute-backend  default     N/A       fork    8273   0s        0    online    0%    24.4mb  ubuntu  disabled

ubuntu@ip-172-31-45-106:/var/www/freshroute-api/Backend$ pm2 status

id   name           namespace    version    mode    pid    uptime    Ⓢ    status    cpu    mem    user    watching
0    freshroute-backend  default     N/A       fork    8273   8s        0    online    0%    53.0mb  ubuntu  disabled

ubuntu@ip-172-31-45-106:/var/www/freshroute-api/Backend$ pm2 logs 0
[TAILING] Tailing last 15 lines for [0] process (change the value with --lines option)
/home/ubuntu/.pm2/logs/freshroute-backend-error.log last 15 lines:
/home/ubuntu/.pm2/logs/freshroute-backend-out.log last 15 lines:
0|freshrou |
0|freshrou | > logistic-backend@1.0.0 start
0|freshrou | > node index.js
0|freshrou |
0|freshrou | [dotenv@17.2.3] injecting env (0) from .env -- tip: ⚙ enable debug logging with { debug: true }
0|freshrou | FreshRoute backend running

```

- Nginx status and nginx -t success output

```

GNU nano 8.7.1 /etc/nginx/sites-available/freshroute-api
server {
    listen 80 default_server;
    server_name api.freshroute.lk 15.206.81.206 _;

    location / {
        proxy_pass http://127.0.0.1:4000;

        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;

        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

- Public API health endpoint response.

```

<  →  ↻  ⚠ Not secure 15.206.81.206/health
Pretty print
{"status":"Healthy","message":"FreshRoute backend running"}

```

- GitHub Actions successful deployment run.

The screenshot shows the GitHub Actions interface for the repository 'AashaniSamarakoon / RP_FreshRoute_Backend'. The 'Actions' tab is selected, displaying a list of workflow runs. The workflow 'Deploy Freshroute Backend' is highlighted, showing two successful runs. The first run, labeled 'health check update', was triggered by a commit push and completed successfully at 6:02 PM. The second run, also labeled 'health check update', was triggered by a deployment event and completed successfully at 6:04 PM. Both runs show a status of 'deployment' and a duration of approximately 16 seconds.

Appendix D: Reference Documentation

- AWS EC2 connection and key pairs: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connect.html>
- AWS CloudWatch Agent: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Install-CloudWatch-Agent.html>
- React Native publishing to Google Play: <https://reactnative.dev/docs/signed-apk-android.html>
- Expo EAS Build: <https://docs.expo.dev/build/introduction/>
- Expo production builds: <https://docs.expo.dev/deploy/build-project/>